ISLAMIC UNIVERSITY OF TECHNOLOGY

DHAKA, BANGLADESH

ORGANIZATION OF ISLAMIC COOPERATION

DATABASE MANAGEMENT SYSTEMS LAB

Name: Antara Arifa Mullick

Student ID: 210042132

Department: Computer Science and Engineering

Program: B.Sc. in Software Engineering

Report preparing for Lab Task

Here is the problem statement -

Execute the banking.sql script using command. It creates a set of tables along with values that

maintain the following schema:

Here, the boldfaces denote the primary keys and the arcs denote the foreign key relationships.

In this lab, you have to write all SQL statements in an editor first and save them with .sql extension.

Then execute the SQL script.

Write SQL statements for the following queries:

1.Find all customer names and their cities who have a loan but not an account.

Solution Approach

```
select distinct
c.customer_name,c.customer_city
from customer c
join borrower b on
c.customer_name= b.customer_name
where c.customer_name NOT
in(select distinct customer_name
from depositor
 );
```

SQL Command

```
SQL> select distinct
  2  c.customer_name,c.customer_city
  3  from customer c
  4  join borrower b on
  5  c.customer_name= b.customer_name
  6  where c.customer_name NOT
  7  in(select distinct customer_name
  8  from depositor
  9   );

CUSTOMER_NAME   CUSTOMER_CITY
--------------- ---------------
Jackson         Salt Lake
Adams           Pittsfield
McBride         Rye
Curry           Rye
```

2.Find all customer names who have an account as well as a loan.

Solution Approach

```
SELECT CUSTOMER_NAME
FROM CUSTOMER
WHERE CUSTOMER_NAME IN( SELECT CUSTOMER_NAME FROM
BORROWER)AND
CUSTOMER_NAME IN(
SELECT CUSTOMER_NAME FROM DEPOSITOR
);
```

SQL Command

```
SQL> SELECT CUSTOMER_NAME
  2  FROM CUSTOMER
  3  WHERE CUSTOMER_NAME IN( SELECT CUSTOMER_NAME FROM BORROWER)AND
  4  CUSTOMER_NAME IN(
  5  SELECT CUSTOMER_NAME FROM DEPOSITOR
  6  );

CUSTOMER_NAME
---------------
Jones
Smith
Hayes
```

3. Show the count of accounts that were opened in each month along with the month.

Solution Approach

```
SELECT TO_CHAR(acc_opening_date, 'MM') AS month,
COUNT(*) AS account_count
FROM account
GROUP BY TO_CHAR(acc_opening_date, 'MM')
ORDER BY TO_CHAR(acc_opening_date, 'MM');
```

SQL Command

```
SQL> SELECT TO_CHAR(acc_opening_date, 'MM') AS month, COUNT(*) AS account_count
  2  FROM account
  3  GROUP BY TO_CHAR(acc_opening_date, 'MM')
  4  ORDER BY TO_CHAR(acc_opening_date, 'MM');

MO ACCOUNT_COUNT
-- -------------
01              1
03              2
04              1
05              1
07              1
08              1
09              1
11              1

8 rows selected.
```

4. Find the months between the last acc_opening_date and last loan_date of customer 'Smith'.

Solution Approach

```
select months_between(
(select max(acc_opening_date)
from account
where account_number in(
select account_number
from depositor
where customer_name = 'Smith')),
(select max(loan_date)
from loan
where loan_number in(
select loan_number
```

```
from borrower
where customer_name = 'Smith'
)))
as month
from dual;
```

SQL Command

```
SQL> select months_between(
  2  (select max(acc_opening_date)
  3  from account
  4  where account_number in(
  5  select account_number
  6  from depositor
  7  where customer_name = 'Smith')),
  8  (select max(loan_date)
  9  from loan
 10  where loan_number in(
 11  select loan_number
 12  from borrower
 13  where customer_name = 'Smith'
 14  )))
 15  as month
 16  from dual;

     MONTH
----------
-3.7741935
```

5. Find the average loan amount at each branch. Do not include any branch which is located in a that has the substring, ’Horse’

in its name.

Solution Approach

```
SELECT B.BRANCH_NAME, AVG(L.AMOUNT) AS
AVERAGE_LOAN_AMOUNT
FROM BRANCH B
JOIN LOAN L ON B.BRANCH_NAME = L.BRANCH_NAME
WHERE B.BRANCH_CITY NOT LIKE '%HORSE%'
GROUP BY B.BRANCH_NAME;
```

SQL Command

```
SQL> SELECT B.BRANCH_NAME, AVG(L.AMOUNT) AS
  2  AVERAGE_LOAN_AMOUNT
  3  FROM BRANCH B
  4  JOIN LOAN L ON B.BRANCH_NAME = L.BRANCH_NAME
  5  WHERE B.BRANCH_CITY NOT LIKE '%HORSE%'
  6  GROUP BY B.BRANCH_NAME;

BRANCH_NAME     AVERAGE_LOAN_AMOUNT
--------------- -------------------
Round Hill                      900
Mianus                          500
Perryridge                     1400
Redwood                        2000
Central                         570
Downtown                       1250
North Town                     7500

7 rows selected.
```

6. Find the customer name and account number of the account that has the highest

balance.

Solution Approach

```
SELECT D.CUSTOMER_NAME, D.ACCOUNT_NUMBER,
A.BALANCE FROM DEPOSITOR D
JOIN ACCOUNT A ON D.ACCOUNT_NUMBER = A.ACCOUNT_NUMBER
WHERE A.BALANCE =
(SELECT MAX(BALANCE) FROM ACCOUNT
);
```

SQL Command

```
SQL> SELECT D.CUSTOMER_NAME, D.ACCOUNT_NUMBER,
  2  A.BALANCE FROM DEPOSITOR D
  3  JOIN ACCOUNT A ON D.ACCOUNT_NUMBER = A.ACCOUNT_NUMBER WHERE A.BALANCE =
  4  (SELECT MAX(BALANCE) FROM ACCOUNT
  5  );

CUSTOMER_NAME   ACCOUNT_NUMBER     BALANCE
--------------- --------------- ----------
Johnson         A-201                9000
```

7. For each branch city, find the average amount of all the loans opened in a branch located in that branch city. Do not include any branch city in the result where the average amount of all

loans opened in a branch located in that city is less than 1500.

Solution Approach

```
SELECT B.BRANCH_CITY,
AVG(L.AMOUNT) AS AVERAGE_CITY_LOAN FROM BRANCH B
JOIN LOAN L ON B.BRANCH_NAME = L.BRANCH_NAME GROUP BY
B.BRANCH_CITY HAVING
AVG(L.AMOUNT) >= 1500;
```

SQL Command

```
SQL> SELECT B.BRANCH_CITY,
  2  AVG(L.AMOUNT) AS AVERAGE_CITY_LOAN FROM BRANCH B
  3  JOIN LOAN L ON B.BRANCH_NAME = L.BRANCH_NAME GROUP BY B.BRANCH_CITY HAVING
  4  AVG(L.AMOUNT) >= 1500;

BRANCH_CITY     AVERAGE_CITY_LOAN
--------------- -----------------
Palo Alto                  2000
Rye                        4035
```

8.Show all the name of the customer with the suffix 'Eligible' who has at least one loan that can be paid off by his/her total balance.

Solution Approach

```
select customer_name || ' Eligible'
as customer_name from depositor
where account_number in
(select account_number from account
where balance >=
(select sum(amount) from loan
where loan.branch_name = account.branch_name
and loan.loan_number in
(select loan_number from borrower
where borrower.customer_name =
depositor.customer_name)
)
);
```

SQL Command

```
SQL> select customer_name || ' Eligible'
  2  as customer_name from depositor
  3  where account_number in
  4  (select account_number from account
  5  where balance >=
  6  (select sum(amount) from loan
  7  where loan.branch_name = account.branch_name
  8  and loan.loan_number in
  9  (select loan_number from borrower
 10  where borrower.customer_name = depositor.customer_name)
 11  )
 12  );

CUSTOMER_NAME
----------------------
Hayes Eligible
```

9. Show all the branch names with suffixes

’Elite’ that have a total account balance greater than the (average total balance + 500), ’Moderate’ that have a total account balance in between (average total balance + 500) to (average total balance - 500), else ’Poor’.

Solution Approach

```
Show all the branch names with suffixes ’Elite’ that
have a total account balance greater than the
(average total balance + 500), ’Moderate’ that have a
total account balance in between (average total
balance + 500) to (average total balance - 500), else
’Poor’.
SELECT branch_name || ' Elite' AS branch_name from
branch
where branch_name in
(
    select branch_name from account
    group by branch_name
    having sum(balance) >
    (
        select avg(sum_balance) + 500 from
        (
```

```
            select branch_name, sum(balance) as
sum_balance from account
            group by branch_name
        )
    )
)
UNION
SELECT branch_name || ' Moderate' AS branch_name from
branch
where branch_name in
(
    select branch_name from account
    group by branch_name
    having sum(balance) between
    (
        select avg(sum_balance) + 500 from
        (
            select branch_name, sum(balance) as
sum_balance from account
            group by branch_name
        )
    )
    and
    (
        select avg(sum_balance) - 500 from
        (
            select branch_name, sum(balance) as
sum_balance from account
            group by branch_name
        )
    )
)
UNION
```

```
SELECT branch_name || ' Poor' AS branch_name from
branch
where branch_name in
(
    select branch_name from account
    group by branch_name
    having sum(balance) <
    (
        select avg(sum_balance) - 500 from
        (
            select branch_name, sum(balance) as
sum_balance from account
            group by branch_name
        )
    )
);
```

SQL Command

```
BRANCH_NAME
----------------------
Central Poor
Downtown Poor
Mianus Poor
North Town Poor
Perryridge Elite
Redwood Poor
Round Hill Poor

7 rows selected.
```

10. Find the branch information for cities where at least one customer lives who

does not have any account or any loans. The branch must have given some loans and has accounts opened by other customers.

Solution Approach

```
SELECT branch_name, branch_city from branch
where branch_city in
(
    select customer_city from customer
    where customer_city not in
    (
        select customer_city from customer
        where customer_name in
        (
            select customer_name from depositor
        )
        or
        customer_name in
        (
            select customer_name from borrower
        )
    )
)
and branch_name in
(
    select branch_name from loan
    group by branch_name
    having count(*) > 0
```

```
)
and branch_name in
(
    select branch_name from account
    group by branch_name
    having count(*) > 0
);
```

SQL Command

```
SQL> SELECT branch_name, branch_city from branch
  2  where branch_city in
  3  (
  4      select customer_city from customer
  5      where customer_city not in
  6      (
  7          select customer_city from customer
  8          where customer_name in
  9          (
 10              select customer_name from depositor
 11          )
 12          or
 13          customer_name in
 14          (
 15              select customer_name from borrower
 16          )
 17      )
 18  )
 19  and branch_name in
 20  (
 21      select branch_name from loan
 22      group by branch_name
 23      having count(*) > 0
 24  )
 25  and branch_name in
 26  (
 27      select branch_name from account
 28      group by branch_name
 29      having count(*) > 0
 30  );

BRANCH_NAME     BRANCH_CITY
--------------- ---------------
Downtown        Brooklyn
```

11.Create a new customer_new table using a similar structure to the

customer table.

Solution Approach

```
CREATE TABLE customer_new AS
SELECT * FROM customer
where customer_name = 'dog';
```

SQL Command

```
SQL> CREATE TABLE customer_new AS
  2  SELECT * FROM customer
  3  where customer_name = 'dog';

Table created.
```

12. In the customer_new table insert only those customers who have either an account or a loan.

Solution Approach

```
INSERT INTO customer_new
SELECT * FROM customer
where customer_name in
(
```

```
    select customer_name from depositor
)
or
customer_name in
(
    select customer_name from borrower
);
```

SQL Command

```
SQL> INSERT INTO customer_new
  2  SELECT * FROM customer
  3  where customer_name in
  4  (
  5      select customer_name from depositor
  6  )
  7  or
  8  customer_name in
  9  (
 10      select customer_name from borrower
 11  );

11 rows created.
```

13.Add a new column Status in customer_new table of varchar2(15) type.

Solution Approach

```
ALTER TABLE customer_new
ADD status VARCHAR2(15);
```

SQL Command

```
SQL> ALTER TABLE customer_new
  2  ADD status VARCHAR2(15);

Table altered.
```

14. For each customer if his/her total balance is greater than the total loan then set the status 'In savings', if the vise versa then 'In loan', lastly if both of the amounts are the same then 'In Breakeven'.

Solution Approach

```
UPDATE customer_new
SET Status = (
    CASE
        WHEN (
            SELECT SUM(a.balance)
            FROM account a
            WHERE EXISTS (
                SELECT 1
                FROM depositor d
                WHERE d.customer_name =
customer_new.customer_name
                AND d.account_number =
a.account_number
```

```
            )
        ) > (
            SELECT SUM(l.amount)
            FROM loan l
            WHERE EXISTS (
                SELECT 1
                FROM borrower b
                WHERE b.customer_name =
customer_new.customer_name
                AND b.loan_number = l.loan_number
            )
        ) THEN 'In Savings'
        WHEN (
            SELECT SUM(a.balance)
            FROM account a
            WHERE EXISTS (
                SELECT 1
                FROM depositor d
                WHERE d.customer_name =
customer_new.customer_name
                AND d.account_number =
a.account_number
            )
        ) < (
            SELECT SUM(l.amount)
            FROM loan l
            WHERE EXISTS (
                SELECT 1
                FROM borrower b
                WHERE b.customer_name =
customer_new.customer_name
                AND b.loan_number = l.loan_number
            )
        ) THEN 'In Loan'
```

```
        ELSE 'In Breakeven'
    END
);
```

SQL Command

```
SQL> UPDATE customer_new
  2  SET Status = (
  3      CASE
  4          WHEN (
  5              SELECT SUM(a.balance)
  6              FROM account a
  7              WHERE EXISTS (
  8                  SELECT 1
  9                  FROM depositor d
 10                  WHERE d.customer_name = customer_new.customer_name
 11                  AND d.account_number = a.account_number
 12              )
 13          ) > (
 14              SELECT SUM(l.amount)
 15              FROM loan l
 16              WHERE EXISTS (
 17                  SELECT 1
 18                  FROM borrower b
 19                  WHERE b.customer_name = customer_new.customer_name
 20                  AND b.loan_number = l.loan_number
 21              )
 22          ) THEN 'In Savings'
 23          WHEN (
 24              SELECT SUM(a.balance)
 25              FROM account a
 26              WHERE EXISTS (
 27                  SELECT 1
 28                  FROM depositor d
 29                  WHERE d.customer_name = customer_new.customer_name
 30                  AND d.account_number = a.account_number
 31              )
 32          ) < (
 33              SELECT SUM(l.amount)
 34              FROM loan l
 35              WHERE EXISTS (
 36                  SELECT 1
 37                  FROM borrower b
 38                  WHERE b.customer_name = customer_new.customer_name
 39                  AND b.loan_number = l.loan_number
 40              )
 41          ) THEN 'In Loan'
 42          ELSE 'In Breakeven'
 43      END
 44  );
UPDATE customer_new
```

15. Count the occurrences of each status type in customer_new table.

Solution Approach

```
SELECT status, COUNT(*) as status_count
FROM customer_new
GROUP BY status;
```

SQL Command

```
SQL> SELECT status, COUNT(*) as status_count
  2  FROM customer_new
  3  GROUP BY status;

STATUS          STATUS_COUNT
--------------- ------------
                          11
```

Problems faced :

Syntax Errors: SQL queries must have some strict syntax requirements. Incorrectly worded queries might result in syntax problems and as a beginner I did struggle to get the syntax properly.

Understanding Database Structure: As I was not so familiar with the structure of the database I am working with, it might be

difficult to determine which tables exist, what columns they include, and how they relate to one another.

Table Creation: Creating tables with the right structure and data types were a bit tricky. Defining primary keys,**Foreign keys**, and constraints correctly is essential.

Connect Table : to connect multiple table command is a bit difficult to me.